

PRACTICAL LIST - DWM

DWM

PRACTICAL LIST – DWM

WEKA: Knowledge Flow

1. Decision Tree (Classification)
2. Naive Bayesian (Classification)
3. Simple K-Means (Clustering)
4. Agglomerative (Hierarchical Clustering)
5. Apriori (Association Rule Mining)
6. FP Growth (Association Rule Mining)

CODES:

- a. OLAP (on Oracle MYSQL)
- b. Decision Tree (Classification)
- c. Naive Bayesian (Classification)
- d. K-means – 1D & 2D K-means (Clustering)
- e. Agglomerative – Single, Complete & Average (Hierarchical Clustering)
- f. Apriori (Association Rule Mining)
- g. FP Tree (Association Rule Mining)
- h. Linear Regression (Prediction Algorithm)
- i. Data Discretization & Data Visualization

OLAP Problem Statements

OLAP Problem Statements:

1. Problem Statement:

This project designs a Data Warehouse for hospital visits using Star and Snowflake Schemas to analyze doctor consultations, patient visits, and revenue trends. The Fact Table stores visit details along with measures like count (number of visits) and charges (consultation fees). The Dimension Tables include Doctor, Patient, and Time data.

2. Problem Statement:

Design a data warehouse for a regional weather bureau. The weather bureau has about 100 probes, which are scattered throughout various land and ocean locations in the region to collect basic weather data, including air pressure, temperature and precipitation at each hour. All data is sent to the central station, which has collected such data for more than 10 years.

3. Problem Statement:

An Auction company wants to design a data warehouse to record the sold price of items with their low estimate, high estimate and reserve price. There are four dimensions - Item, Consigner, Buyer and Time. Explain all aspects of the diagram. Design Star and Snowflake schema for above case.

4. Problem Statement:

Suppose a company wants to design a data warehouse to facilitate the analysis of moving vehicles in an online analytical processing manner. The company registers a huge amount of auto movement data in the format of (auto id, location, speed, time). Each auto id represents a vehicle associated with information (example: vehicle, category, driver category) and each location may be associated with a street in the city. assume that a street map is available for the city. Design data warehouse to facilitate effective online analytical processing in multidimensional space. Also Draw Star and Snowflake Schema for the same

5. Problem Statement:

Electronics company has a sales department.

Consider 4 dimensions - Time, Product, Branch and location.

The schema contains a central fact table sales with two measures, dollars sold and units sold.

6. Problem Statement:

Given information package for recording information requirement for "Hotel Occupancy". Considering dimensions like Time, Hotel etc. Design Star schema and Snowflake schema for above case with IPD.

7. Problem Statement:

Bank wants to develop a data warehouse for affecting decision making about their loan schema. The Bank provides loans to customers for various Purposes like house building loan, car loan, education loan, Personal loan etc.. The whole Country is categorized into a number of regions viz. North, South, East and West. Each region consists of a set of status. Loan is distributed to customers at an interest rate that changes from time to time. Also at a given Point of time, the different types of loan have different rates. The data warehouse should record an entity for each disbursement of loan to customer with respect to the above business scenario. Design an IPD clearly. Explain all aspects of the diagram. Design Star and Snowflake schema for above case.

. . .

8. Problem Statement:

The Mumbai University wants to help to design a star schema to record grades for courses completed by students. There are 4 dimensions tables: Course section, Professor, Student & Period with attributes as follows.

1. Course sections: Course_ID, Section_ID, Course_Name, Units, Room_ID, Room_Capacity.
During a given semester the college offers an average of 500 Course Sections.
2. Professor: Professor_ID, Professor_Name, Title, Department_ID, Department_Name
3. Student: Student_ID, Student_Major.
Each course section has an average of 60 students.
4. Period: Semester_ID, Year.

It will contain data for a 30 month period. The only fact that is to be recorded in the fact table is course grade. Design Star schema and Snowflake schema for above case.

9. Problem Statement:

An All Electronics Company has a sales department. Consider four dimensions: Time, Product, Branch & Location. The schema contains a central fact table sale with two measures: dollars sold and units sold. Design Star Schema & Snowflake Schema for above case.

DWM - SUMS

DWM - SUMS

1. **Binning:**

- Step 1: Partition the sorted values in equal numbers of bins.
- Step 2: Smoothing done by bin means
- Step 3: Smoothing done by bin boundaries

2. **Correlation Analysis:**

$$\text{cov}(a, b) = \frac{\sum(a_i \cdot b_i)}{n} - \bar{a} \cdot \bar{b}$$

This formula is used to calculate Covariance (A statistical measure that describes the relationship between two variables)

If $\text{cov}(a, b) > 0$ then as 'a' increases, 'b' also increases

If $\text{cov}(a, b) < 0$ then as 'a' decreases, 'b' also decreases

If $\text{cov}(a, b) = 0$ then there is no linear relationship between 'a' & 'b'

3. **Classification → Decision Tree**

check or ask for notes

4. **Classification → Naive Bayesian**

- Step 1: Calculate Prior Probability for each Class
- Step 2: Calculate Conditional Probabilities based on the Sample X to be predicted
- Step 3: Calculate the Likelihood by multiplying the Conditional Probabilities for the respective classes
- Step 4: Multiply Likelihood & Prior Probability for each class to get Posterior Probability
- Step 5: Compare and make conclusion

5. **Linear Regression**

- Step 1: Calculate Avg(x) & Avg(y)
- Step 2: Calculate $(x_i - \text{Avg}(x)), (x_i - \text{Avg}(x))^2$ and $(y_i - \text{Avg}(y))$
- Step 3: Calculate Beta(B)
 $B = \frac{\text{Summation of } [(x_i - \text{Avg}(x)) \text{ multiply } (y_i - \text{Avg}(y))]}{[\text{Summation of } (x_i - \text{Avg}(x))^2]}$
- Step 4: Calculate alpha = Avg(y) - B(Avg(x))
- Step 5: Calculate Linear Regression Equation = $y = \text{alpha} + Bx$ (value of x will be given in the question)

Beta is the slope. It tells us how much y increases or decreases when x is increased by 1 unit

Alpha is the intercept. It is the baseline/starting value. It is the predicted value of y when $x = 0$

6. Partitioning Clustering → K Means - 1D

Step 1: Assume number of clusters (k) = n

Step 2: Assume means m_1 and m_2 from the dataset itself.

Step 3: Based on the means m_1 and m_2 define cluster k_1 and k_2 depending upon which points are closest in the dataset.

Step 4: New m_1 & m_2 is the average of clusters k_1 and k_2 .

Step 5: Repeat the process until there is no change in the clusters.

7. Partitioning Clustering → K Means - 2D

Step 0: Assuming we have to create 2 clusters

Step 1: Find adjacency matrix using Euclidean Distance Formula

Step 2: Let 2 points be centroids (Ex : Point A & Point C)

Step 3: Assign each object based on the minimum distance, thus finding G^i (where $i = 0, 1, 2, \dots$) and subsequently make clusters Group 1, Group 2

Step 4: Now centroids c_1 and c_2 are the averages of members of Group 1 and Group 2 clusters

Step 5: Calculate distance again based on c_1 & c_2

Step 6: Repeat from Step 3 until clusters keep having the same members

We already know about the number of clusters needed. Used in case of large data. Faster technique. Less flexible.

8. Hierarchical Clustering → Agglomerative (Single Linkage, Complete Linkage, Average Linkage)

Step 0: Create adjacency matrix using Euclidean Distance Formula

Step 1: Find the smallest/lowest value which will be merged

Step 2: Show the merged clusters on a dendrogram & recompute the distances. Use MIN, MAX or AVG based upon single, complete or average linkage.

Step 3: Repeat from Step 1 until all clusters are merged

Each independent value is its own cluster at the beginning. It's a bottom up approach. Closest clusters are merged depending on linkages and not centroids. Number of clusters is not known from the beginning. Used for smaller datasets. Slower technique. More flexible.

All individual points are merged into 1 big cluster. The nearest points on the dendrogram merged first and are closely related whereas the far away points merged later and are not so related.

9. **Association** → **Apriori**

check or ask for notes

10. **Association** → **FP Growth**

check or ask for notes

(Topics in Underlined are Important & First Priority)

DWM - WEKA

DWM – WEKA (Understand what you're doing!)

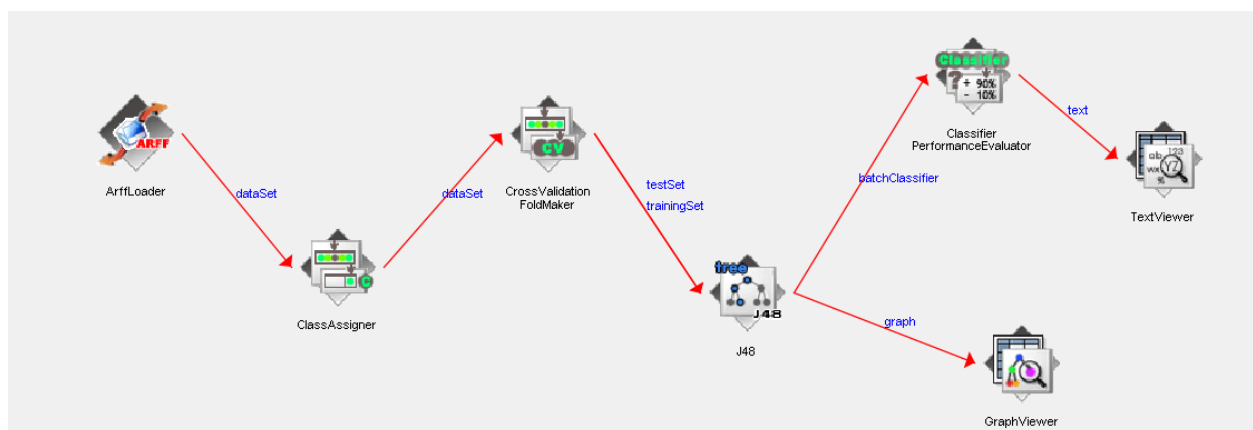
1. Decision Tree (Classification)

STEPS:

- 1) Open Knowledge Flow
- 2) Data Sources → ARFF Loader
- 3) Evaluation → ClassAssigner & CrossValidationFoldMaker
- 4) Classifiers → trees → J48
- 5) Evaluation → ClassifierPerformanceEvaluator
- 6) Visualization → GraphViewer & TextViewer
- 7) RC ARFF Loader → configure → browse → data → iris
- 8) RC ARFF Loader → dataSet → ClassAssigner
- 9) RC ClassAssigner → dataSet → CrossValidationFoldMaker
- 10) RC CrossValidationFoldMaker → trainingSet + testSet → J48
- 11) RC J48 → batchClassifier → ClassifierPerformanceEvaluator
- 12) RC J48 → graph → GraphViewer
- 13) Run
- 14) RC ClassifierPerformanceEvaluator → text → TextViewer
- 15) GraphViewer → Show plots & TextViewer → Show results

(To Change the number of Sets/Folds, go to CrossValidationFoldMaker → Right Click → configure → Number of folds = n → OK)

(RC means RIGHT CLICK)



```
=== Evaluation result ===

Scheme: J48
Options: -C 0.25 -M 2
Relation: iris

=== Summary ===

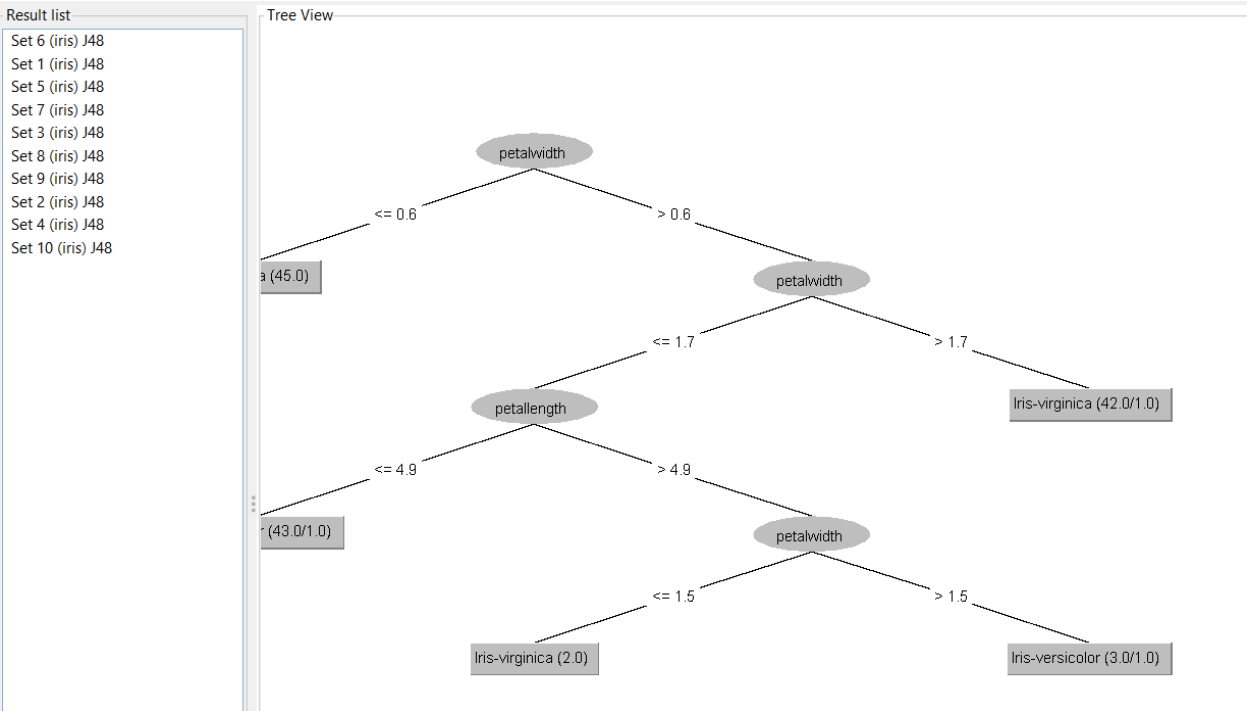
Correctly Classified Instances      144          96      %
Incorrectly Classified Instances    6           4      %
Kappa statistic                    0.94
Mean absolute error                 0.035
Root mean squared error            0.1586
Relative absolute error             7.8705 %
Root relative squared error        33.6353 %
Total Number of Instances          150

=== Detailed Accuracy By Class ===

      TP Rate  FP Rate  Precision  Recall  F-Measure  MCC      ROC Area  PRC Area  Class
      0.980    0.000    1.000     0.980   0.990     0.985   0.990    0.987   Iris-setosa
      0.940    0.030    0.940     0.940   0.940     0.910   0.952    0.880   Iris-versicolor
      0.960    0.030    0.941     0.960   0.950     0.925   0.961    0.905   Iris-virginica
Weighted Avg.   0.960    0.020    0.960     0.960   0.960     0.940   0.968    0.924

=== Confusion Matrix ===

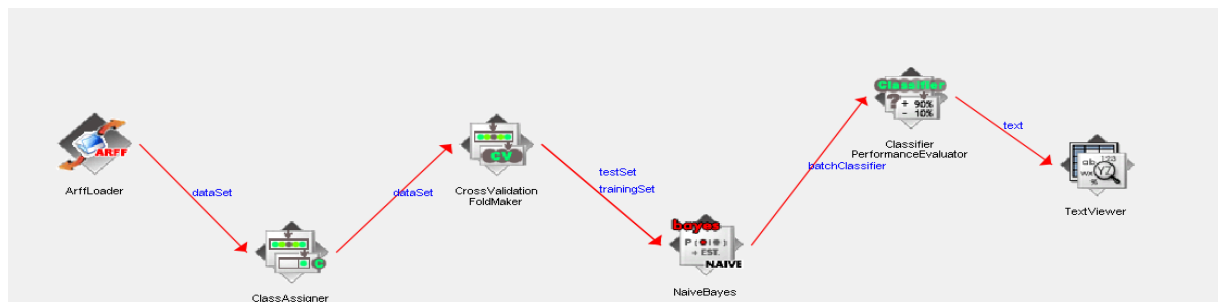
  a  b  c  <-- classified as
49  1  0 | a = Iris-setosa
 0 47  3 | b = Iris-versicolor
 0  2 48 | c = Iris-virginica
```



2. Naive Bayesian (Classification)

STEPS:

- 1) Open Knowledge Flow
- 2) Data Sources → ARFF Loader
- 3) Evaluation → ClassAssigner & CrossValidationFoldMaker
- 4) Classifiers → bayes → NaiveBayes
- 5) Evaluation → ClassifierPerformanceEvaluator
- 6) Visualization → TextViewer
- 7) RC ARFF Loader → configure → browse → data → iris
- 8) RC ARFF Loader → dataSet → ClassAssigner
- 9) RC ClassAssigner → dataSet → CrossValidationFoldMaker
- 10) RC CrossValidationFoldMaker → trainingSet + testSet → NaiveBayes
- 11) RC NaiveBayes → batchClassifier → ClassifierPerformanceEvaluator
- 13) Run
- 14) RC ClassifierPerformanceEvaluator → text → TextViewer
- 15) TextViewer → Show results



=== Evaluation result ===

Scheme: NaiveBayes
Relation: iris

=== Summary ===

Correctly Classified Instances	144	96	%
Incorrectly Classified Instances	6	4	%
Kappa statistic	0.94		
Mean absolute error	0.0342		
Root mean squared error	0.155		
Relative absolute error	7.6997 %		
Root relative squared error	32.8794 %		
Total Number of Instances	150		

=== Detailed Accuracy By Class ===

	TP Rate	FP Rate	Precision	Recall	F-Measure	MCC	ROC Area	PRC Area	Class
	1.000	0.000	1.000	1.000	1.000	1.000	1.000	1.000	Iris-setosa
	0.960	0.040	0.923	0.960	0.941	0.911	0.992	0.983	Iris-versicolor
	0.920	0.020	0.958	0.920	0.939	0.910	0.992	0.986	Iris-virginica
Weighted Avg.	0.960	0.020	0.960	0.960	0.960	0.940	0.994	0.989	

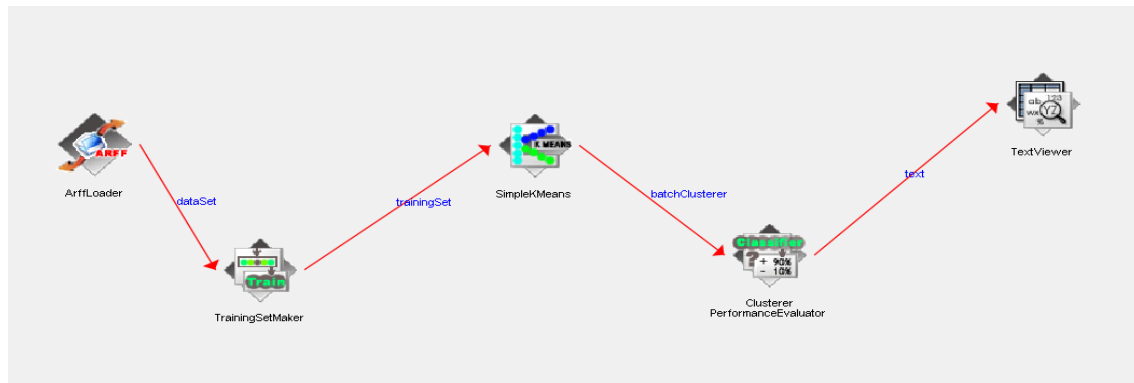
=== Confusion Matrix ===

```
a b c <-- classified as
50 0 0 | a = Iris-setosa
0 48 2 | b = Iris-versicolor
0 4 46 | c = Iris-virginica
```

3. Simple K-Means (Clustering)

STEPS:

- 1) Open Knowledge Flow
- 2) Data Sources → ARFF Loader
- 3) Evaluation → TrainingSetMaker
- 4) Clusters → SimpleKMeans
- 5) Evaluation → ClusterPerformanceEvaluator
- 6) Visualization → TextViewer
- 7) RC ARFF Loader → configure → browse → data → iris
- 8) RC ARFF Loader → dataSet → TrainingSetMaker
- 9) RC TrainingSetMaker → trainingSet → SimpleKMeans
- 10) RC SimpleKMeans → batchClusterer → ClusterPerformanceEvaluator
- 11) RC ClusterPerformanceEvaluator → text → TextViewer
- 12) Run
- 13) TextViewer → Show results



```
=== Evaluation result for training instances ===

Scheme: SimpleKMeans-init 0 -max-candidates 100 -periodic-pruning 10000 -min-density 2.0 -t1 -1.25 -t2 -1.0 -N 2 -A "weka.core.EuclideanDistance" -R first-last" -I 500 -num-slots 1 -S
Relation: iris

kMeans
=====

Number of iterations: 7
Within cluster sum of squared errors: 62.1436882815797

Initial starting points (random):

Cluster 0: 6.1,2.5,4.7,1.4,Iris-versicolor
Cluster 1: 6.2,2.5,4.3,1.3,Iris-versicolor

Missing values globally replaced with mean/mode

Final cluster centroids:

Attribute          Full Data          Cluster#
                   (150.0)          (100.0)          (50.0)
=====
sepal.length       5.8433             6.262            5.006
sepal.width        3.054              2.872            3.418
petal.length       3.7587             4.906            1.464
petal.width        1.1987             1.676            0.244
class              Iris-setosa Iris-versicolor Iris-setosa

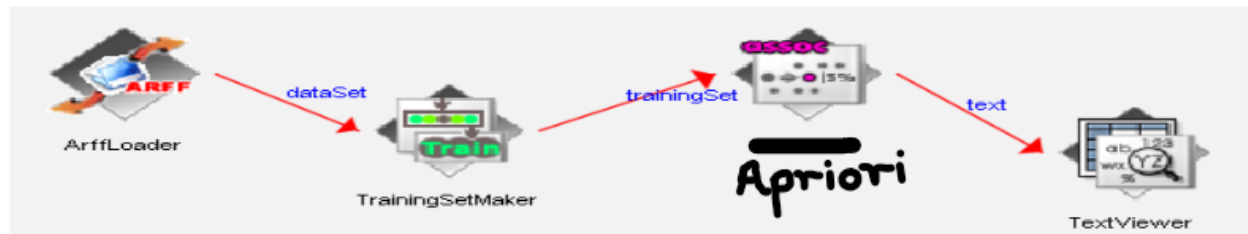
Clustered Instances

0    100 ( 67%)
1     50 ( 33%)
```


5. Apriori (Association Rule Mining)

STEPS:

- 1) Open Knowledge Flow
- 2) Data Sources → ARFF Loader
- 3) Evaluation → TrainingSetMaker
- 4) Associations → Apriori
- 5) Visualization → TextViewer
- 6) RC ARFF Loader → configure → browse → data → supermarket
- 7) RC ARFF Loader → dataSet → TrainingSetMaker
- 8) RC TrainingSetMaker → trainingSet → Apriori
- 9) RC Apriori → text → TextViewer
- 10) Run
- 11) TextViewer → Show results



Result list	Text
11:30:57.973 - Model: Apriori (1)	=== Associator model ===
11:30:57.981 - Model: Apriori (2)	
11:30:57.987 - Model: Apriori (4)	Scheme: Apriori
11:30:57.995 - Model: Apriori (7)	Relation: weather.symbolic
11:30:57.998 - Model: Apriori (5)	
11:30:58.007 - Model: Apriori (9)	Apriori
11:30:58.009 - Model: Apriori (10)	=====
11:30:58.015 - Model: Apriori (8)	Minimum support: 0.2 (2 instances)
11:30:58.016 - Model: Apriori (3)	Minimum metric <confidence>: 0.9
11:30:58.018 - Model: Apriori (6)	Number of cycles performed: 16
	Generated sets of large itemsets:
	Size of set of large itemsets L(1): 12
	Size of set of large itemsets L(2): 41
	Size of set of large itemsets L(3): 28
	Size of set of large itemsets L(4): 5
	Best rules found:
	1. outlook=overcast 4 ==> play=yes 4 <conf:(1)> lift:(1.71) lev:(0.14) [1] conv:(1.67)
	2. temperature=cool 4 ==> humidity=normal 4 <conf:(1)> lift:(2) lev:(0.17) [2] conv:(2)
	3. humidity=normal windy=FALSE 4 ==> play=yes 4 <conf:(1)> lift:(1.71) lev:(0.14) [1] conv:(1.67)
	4. outlook=sunny play=no 3 ==> humidity=high 3 <conf:(1)> lift:(2) lev:(0.13) [1] conv:(1.5)
	5. outlook=sunny humidity=high 3 ==> play=no 3 <conf:(1)> lift:(2.4) lev:(0.15) [1] conv:(1.75)
	6. temperature=cool play=yes 3 ==> humidity=normal 3 <conf:(1)> lift:(2) lev:(0.13) [1] conv:(1.5)
	7. outlook=sunny temperature=hot 2 ==> humidity=high 2 <conf:(1)> lift:(2) lev:(0.08) [1] conv:(1)
	8. temperature=hot play=no 2 ==> outlook=sunny 2 <conf:(1)> lift:(3) lev:(0.11) [1] conv:(1.33)
	9. outlook=sunny temperature=hot 2 ==> play=no 2 <conf:(1)> lift:(2.4) lev:(0.1) [1] conv:(1.17)
	10. windy=FALSE play=no 2 ==> outlook=sunny 2 <conf:(1)> lift:(3) lev:(0.11) [1] conv:(1.33)

6. FP Growth (Association Rule Mining)

STEPS:

- 1) Open Knowledge Flow
- 2) Data Sources → ARFF Loader
- 3) Evaluation → TrainingSetMaker
- 4) Associations → FPGrowth
- 5) Visualization → TextViewer
- 6) RC ARFF Loader → configure → browse → data → supermarket
- 7) RC ARFF Loader → dataSet → TrainingSetMaker
- 8) RC TrainingSetMaker → trainingSet → FPGrowth
- 9) RC FPGrowth → text → TextViewer
- 10) Run
- 11) TextViewer → Show results



```
Result list
11:37:05.809 - Model: FPG

Text
=== Associator model ===

Scheme: FPGrowth
Relation: supermarket

FPGrowth found 16 rules (displaying top 10)

1. [fruit=t, frozen foods=t, biscuits=t, total=high]: 788 ==> [bread and cake=t]: 723 <conf:(0.92)> lift:(1.27) lev:(0.03) conv:(3.35)
2. [fruit=t, baking needs=t, biscuits=t, total=high]: 760 ==> [bread and cake=t]: 696 <conf:(0.92)> lift:(1.27) lev:(0.03) conv:(3.28)
3. [fruit=t, baking needs=t, frozen foods=t, total=high]: 770 ==> [bread and cake=t]: 705 <conf:(0.92)> lift:(1.27) lev:(0.03) conv:(3.27)
4. [fruit=t, vegetables=t, biscuits=t, total=high]: 815 ==> [bread and cake=t]: 746 <conf:(0.92)> lift:(1.27) lev:(0.03) conv:(3.26)
5. [fruit=t, party snack foods=t, total=high]: 854 ==> [bread and cake=t]: 779 <conf:(0.91)> lift:(1.27) lev:(0.04) conv:(3.15)
6. [vegetables=t, frozen foods=t, biscuits=t, total=high]: 797 ==> [bread and cake=t]: 725 <conf:(0.91)> lift:(1.26) lev:(0.03) conv:(3.06)
7. [vegetables=t, baking needs=t, biscuits=t, total=high]: 772 ==> [bread and cake=t]: 701 <conf:(0.91)> lift:(1.26) lev:(0.03) conv:(3.01)
8. [fruit=t, biscuits=t, total=high]: 954 ==> [bread and cake=t]: 866 <conf:(0.91)> lift:(1.26) lev:(0.04) conv:(3)
9. [fruit=t, vegetables=t, frozen foods=t, total=high]: 834 ==> [bread and cake=t]: 757 <conf:(0.91)> lift:(1.26) lev:(0.03) conv:(3)
10. [fruit=t, frozen foods=t, total=high]: 969 ==> [bread and cake=t]: 877 <conf:(0.91)> lift:(1.26) lev:(0.04) conv:(2.92)
```

DWM - CODES

DWM – CODES

Important Libraries to install before starting any Code:

1. Do Windows + R & then write the below command
`pip install pandas numpy matplotlib openpyxl scipy anytree`

Important Instructions:

1. If the code doesn't run on IDLE (due to PC issues in the lab), run it on Jupyter.
2. Run & Practice the codes at least once. Exam mein jaake aisa na hojaye ki samajh hi nahi raha hai
3. Dataset should be imported compulsory! No other codes are allowed! Data will be given in the question. Make an excel file & type the data in it. Excel files are shared in the drive link below. Store the excel file & python code on desktop for simplicity. Copy Excel file's path from its properties & replace it in the code in the line "df = ..."

Cheating Techniques:

1. Google Drive Link for (Pdfs + Word Doc) of codes which can be printed & then cut accordingly (Risky but can be used as Plan B):

<https://drive.google.com/drive/folders/1rY3--vx5seRAtEXbKxNy3V39r-9gO2We?usp=sharing>

2. Go to chrome → shrib.com/#karma → All codes are here.
Copy them and paste it in your notepad in the exam hall.

b. Decision Tree (Classification)

```
import pandas as pd
import math

# Load dataset
df = pd.read_excel(r'C:\Users\Cinepix\Downloads\Electronics.xlsx')

# Drop R_Id if present
if 'R_Id' in df.columns:
    df = df.drop(columns=['R_Id'])

# Target column (assume last)
target_col = df.columns[-1]
features = [col for col in df.columns if col != target_col]

# Function to compute entropy
def entropy(data):
    total = len(data)
    if total == 0:
        return 0
    counts = data[target_col].value_counts()
    ent = 0
    for c in counts:
        p = c / total
        ent -= p * math.log2(p)
    return ent

# Function to compute information gain
def info_gain(data, feature):
    total_entropy = entropy(data)
    values = data[feature].unique()
    weighted_entropy = 0
    for val in values:
        subset = data[data[feature] == val]
        weight = len(subset) / len(data)
        weighted_entropy += weight * entropy(subset)
    return total_entropy - weighted_entropy

# Recursive tree builder
def build_tree(data, features, depth=0):
```

```

labels = data[target_col].unique()

# If only one class remains
if len(labels) == 1:
    return labels[0]

# If no features left
if not features:
    return data[target_col].mode()[0]

# Find best feature
gains = {f: info_gain(data, f) for f in features}
best_feature = max(gains, key=gains.get)

tree = {best_feature: {}}

for val in data[best_feature].unique():
    subset = data[data[best_feature] == val]
    if subset.empty:
        tree[best_feature][val] = data[target_col].mode()[0]
    else:
        remaining_features = [f for f in features if f !=
best_feature]
        tree[best_feature][val] = build_tree(subset,
remaining_features, depth + 1)

return tree

# Pretty print the tree
def print_tree(tree, indent=""):
    if not isinstance(tree, dict):
        print(indent + ">", tree)
        return
    for feature, branches in tree.items():
        for val, subtree in branches.items():
            print(f"{indent}If {feature} == '{val}':")
            print_tree(subtree, indent + " ")

# Build and print the tree
tree = build_tree(df, features)

```

```
print("\nDecision Tree:")  
print_tree(tree)
```


c. Naive Bayesian (Classification)

```
import pandas as pd

#For excel file. Jaha bhi tumhara file stored hai, uska location
df = pd.read_excel(r'C:\Users\Cinepix\Downloads\Electronics.xlsx')

# Assume last column is the class (target)
target_col = df.columns[-1]

```Drop R_Id if present. Excel sheet mein check karlo if there is
serial Number ka such column```
if 'R_Id' in df.columns:
 df = df.drop(columns=['R_Id'])

Get feature columns (excluding the target column)
feature_cols = [col for col in df.columns if col != target_col]

Step 1: Prior probabilities
total_samples = len(df)
classes = df[target_col].unique()
priors = {c: len(df[df[target_col] == c]) / total_samples for c in
classes}

print("\nPrior Probabilities:")
for c in priors:
 print(f"P({target_col} = '{c}') = {round(priors[c], 3)}")

Step 2: Likelihoods
def calc_likelihood(df, feature, feature_value, target_class):
 subset = df[df[target_col] == target_class]
 feature_count = len(subset[subset[feature] == feature_value])
 total_count = len(subset)
 return feature_count / total_count if total_count > 0 else 0

Let's say this is the input sample to predict:
(Change these values based on actual test case)
sample = {
 'Age': 'Youth',
 'Income': 'Medium',
```

```

 'Student': 'Yes',
 'Credit Rating': 'Fair'
 }

Step 3: Calculate likelihoods and posteriors
likelihoods = {}
posteriors = {}

print("\nLikelihoods:")

for c in classes:
 prob = 1
 print(f"\nFor class = '{c}':")
 for feature in feature_cols:
 feature_val = sample.get(feature)
 likelihood = calc_likelihood(df, feature, feature_val, c)
 prob *= likelihood
 print(f"P({feature} = '{feature_val}' | {target_col} = '{c}')")
 = {round(likelihood, 3)}")
 posteriors[c] = prob * priors[c]

Step 4: Print posterior probabilities
print("\nPosterior Probabilities (after applying Bayes' theorem):")
for c in posteriors:
 print(f"P(X | {target_col} = '{c}') * P({target_col} = '{c}') =")
 {round(posteriors[c], 3)}")

Step 5: Predict the class
prediction = max(posteriors, key=posteriors.get)
print(f"\nPredicted class: {target_col} = '{prediction}'")

```

#### **d. K-Means 1D**

```
import pandas as pd
import matplotlib.pyplot as plt
import random

Step 1: Read Excel file
df = pd.read_excel(r'C:\Users\Cinepix\Downloads\KMeans1D.xlsx') #
Replace with actual filename
data = df.iloc[:, 0].tolist() # Get first column as list

Step 2: Set number of clusters
k = int(input("Enter number of clusters (k): ")) # User specifies
number of clusters

Initialize means by randomly selecting k points
means = random.sample(data, k)

def assign_clusters(data, means):
 clusters = [[] for _ in range(len(means))]
 for point in data:
 # Find closest mean
 distances = [abs(point - m) for m in means]
 closest_mean_idx = distances.index(min(distances))
 clusters[closest_mean_idx].append(point)
 return clusters

def compute_mean(cluster):
 return sum(cluster) / len(cluster) if cluster else 0

Step 3: Iterate until means stabilize
for i in range(10): # Limit to 10 iterations max
 clusters = assign_clusters(data, means)
 new_means = [compute_mean(cluster) for cluster in clusters]

 # Check for convergence
 if new_means == means:
 break # Converged

means = new_means
```

```

Step 4: Print results
print("Final clusters:")
for i, cluster in enumerate(clusters, 1):
 print(f"Cluster {i}:", sorted(cluster))
for i, mean in enumerate(means, 1):
 print(f"Mean {i}:", mean)

Step 5: Visualize
colors = ['red', 'blue', 'green', 'purple', 'orange', 'cyan',
'magenta'] # Add more colors if needed
for i, cluster in enumerate(clusters):
 plt.scatter(cluster, [0]*len(cluster), color=colors[i %
len(colors)], label=f'Cluster {i+1}')
plt.scatter(means, [0]*len(means), color='black', label='Centers',
marker='x', s=100)

plt.legend()
plt.title(f"1D K-Means Clustering (k={k})")
plt.yticks([]) # Hide Y-axis since it's 1D
plt.xlabel("Values")
plt.show()

```

#### **d. K-Means 2D**

```
import pandas as pd
import math
import random
import matplotlib.pyplot as plt

Step 1: Load 2D data from Excel

df = pd.read_excel(r'C:\Users\Cinepix\Downloads\KMeans2D.xlsx')
data = df.values.tolist() # Each row is a [x, y] point
Convert lists to tuples to match the first code
data = [tuple(point) for point in data]

Step 2: Get user input for number of clusters
k = int(input("Enter the number of clusters (k): "))
if k > len(data):
 print(f"Error: k ({k}) cannot be larger than number of points ({len(data)})")
 exit()

Step 3: Choose initial centroids (first and third points for k=2,
else random)
if k == 2 and len(data) >= 3:
 centroids = [data[0], data[2]] # First and third points: (2.0,
 2.0) and (1.0, 1.0)
 print("Initial centroids (first and third points):", centroids)
else:
 centroids = random.sample(data, k)
 print("Initial centroids:", centroids)

Step 4: Function to calculate Euclidean distance
def euclidean(p1, p2):
 return math.sqrt((p1[0] - p2[0])**2 + (p1[1] - p2[1])**2)

Step 5: Assign each point to the nearest centroid
def assign_clusters(data, centroids):
 clusters = [[] for _ in range(len(centroids))]
 for point in data:
 distances = [euclidean(point, centroid) for centroid in
 centroids]
```

```

 min_index = distances.index(min(distances))
 clusters[min_index].append(point)
 return clusters

Step 6: Update centroids as the mean of each cluster
def update_centroids(clusters):
 new_centroids = []
 for cluster in clusters:
 if cluster:
 x_coors = [p[0] for p in cluster]
 y_coors = [p[1] for p in cluster]
 new_centroids.append((sum(x_coors)/len(x_coors),
sum(y_coors)/len(y_coors)))
 else:
 new_centroids.append((0, 0)) # Default for empty cluster
 return new_centroids

Step 7: Repeat until convergence or max iterations
def are_centroids_equal(c1, c2, tolerance=1e-6):
 if len(c1) != len(c2):
 return False
 for p1, p2 in zip(c1, c2):
 if not (abs(p1[0] - p2[0]) < tolerance and abs(p1[1] - p2[1])
< tolerance):
 return False
 return True

for iteration in range(10):
 clusters = assign_clusters(data, centroids)
 new_centroids = update_centroids(clusters)
 if are_centroids_equal(new_centroids, centroids):
 print(f"Converged at iteration {iteration+1}!")
 break
 centroids = new_centroids

Step 8: Validate all points are clustered
all_clustered_points = [p for cluster in clusters for p in cluster]
missing_points = [p for p in data if p not in all_clustered_points]
if missing_points:
 print("Error: Missing points:", missing_points)

```

```

else:
 print("All points assigned.")

Step 9: Print final clusters
print("\nFinal clusters:")
for idx, cluster in enumerate(clusters, 1):
 print(f"Cluster {idx} ({len(cluster)} points): {cluster}")

Step 10: Print final centroids
print("\nFinal centroids:")
for idx, centroid in enumerate(centroids, 1):
 print(f"Centroid {idx}: {centroid}")

Step 11: Visualize
colors = ['red', 'blue', 'green', 'purple', 'orange', 'cyan',
'magenta', 'yellow']
plt.figure(figsize=(8, 6))
for i, cluster in enumerate(clusters):
 if cluster:
 x = [p[0] for p in cluster]
 y = [p[1] for p in cluster]
 plt.scatter(x, y, color=colors[i % len(colors)],
label=f'Cluster {i+1}', s=100)
 for point in cluster:
 plt.text(point[0], point[1], f"{point}", fontsize=10,
ha='right')

Plot centroids
mean_x = [m[0] for m in centroids]
mean_y = [m[1] for m in centroids]
plt.scatter(mean_x, mean_y, color='black', marker='x', s=200,
label='Centroids')

plt.legend()
plt.title(f"2D K-Means Clustering (k={k})")
plt.xlabel("X")
plt.ylabel("Y")
plt.grid(True)
plt.show()

```

### **e. Agglomerative - Single, Complete, Average**

```
import pandas as pd
import matplotlib.pyplot as plt
import math
import numpy as np
from scipy.cluster.hierarchy import dendrogram

Step 1: Read Excel file
df = pd.read_excel('Agglomerative.xlsx')
points = df[['X', 'Y']].values
labels = df['Pi'].tolist()

Step 2: Euclidean distance
def euclidean(p1, p2):
 return math.sqrt((p1[0] - p2[0])**2 + (p1[1] - p2[1])**2)

Step 3: Linkage methods
def single_link(c1, c2):
 return min(euclidean(points[i], points[j]) for i in c1 for j in c2)

def complete_link(c1, c2):
 return max(euclidean(points[i], points[j]) for i in c1 for j in c2)

def average_link(c1, c2):
 dists = [euclidean(points[i], points[j]) for i in c1 for j in c2]
 return sum(dists) / len(dists)

Step 4: Agglomerative clustering with stepwise cluster output
def agglomerative_clustering(link_func, method_name):
 clusters = [[i] for i in range(len(points))]
 cluster_labels = [[labels[i]] for i in range(len(points))]
 history = []
 label_map = {i: i for i in range(len(points))}
 current_cluster_id = len(points)

 # Step 0: Initial cluster state
 print(f"\nInitial Clusters ({method_name} Linkage):")
```



```

print([cl for cl in cluster_labels])

step = 1
while len(clusters) > 1:
 min_dist = float('inf')
 to_merge = (0, 1)

 for i in range(len(clusters)):
 for j in range(i + 1, len(clusters)):
 dist = link_func(clusters[i], clusters[j])
 if dist < min_dist:
 min_dist = dist
 to_merge = (i, j)

 i, j = to_merge
 c1, c2 = clusters[i], clusters[j]
 label1, label2 = cluster_labels[i], cluster_labels[j]
 new_cluster = c1 + c2
 new_label = label1 + label2

 id1 = label_map[c1[0]]
 id2 = label_map[c2[0]]
 history.append([id1, id2, min_dist, len(new_cluster)])

 for idx in new_cluster:
 label_map[idx] = current_cluster_id
 current_cluster_id += 1

 # Update clusters and print
 clusters = [clusters[k] for k in range(len(clusters)) if k not
in to_merge]
 cluster_labels = [cluster_labels[k] for k in
range(len(cluster_labels)) if k not in to_merge]
 clusters.append(new_cluster)
 cluster_labels.append(new_label)

 print(f"\nStep {step}:")
 print([sorted(cl) for cl in cluster_labels])
 step += 1

```

```

 return np.array(history)

Step 5: Dendrogram plotting
def plot_dendrogram(history, method_name):
 plt.figure(figsize=(10, 6))
 dendrogram(history, labels=labels)
 plt.title(f"Agglomerative Clustering Dendrogram ({method_name}
Linkage)")
 plt.xlabel("Points")
 plt.ylabel("Distance")
 plt.grid(True)
 plt.tight_layout()
 plt.show()

Step 6: Method selection
def select_method():
 print("Select linkage method:")
 print("1. Single Linkage")
 print("2. Complete Linkage")
 print("3. Average Linkage")
 choice = input("Enter 1, 2, or 3: ")

 if choice == '1':
 return "Single", single_link
 elif choice == '2':
 return "Complete", complete_link
 elif choice == '3':
 return "Average", average_link
 else:
 print("Invalid choice. Defaulting to Single Linkage.")
 return "Single", single_link

Step 7: Run the program
method_name, func = select_method()
history = agglomerative_clustering(func, method_name)
plot_dendrogram(history, method_name)

```

## **f. Apriori**

```
import pandas as pd
from itertools import combinations

--- Load Excel File ---
df = pd.read_excel(r'C:\Users\Cinepix\Downloads\Apriori.xlsx')

--- Preprocess Transactions ---
transactions = df['List of item Ids'].dropna().apply(lambda x:
set(str(x).strip().lower().split(' '))).tolist()
num_transactions = len(transactions)

--- User Input ---
min_support_percent = float(input("Enter minimum support %: "))
min_confidence_percent = float(input("Enter minimum confidence %: "))

--- Step 1: Minimum Support Count ---
min_support_count = round((min_support_percent * num_transactions) /
100)
print(f"\nMinimum Support Count: {min_support_count}")

--- Support Count Helper ---
def count_support(itemset, transactions):
 return sum(1 for t in transactions if itemset.issubset(t))

--- Step 2: Generate C1 and L1 ---
item_counts = {}
for transaction in transactions:
 for item in transaction:
 itemset = frozenset([item])
 item_counts[itemset] = item_counts.get(itemset, 0) + 1

C1 = item_counts
L1 = {itemset: count for itemset, count in C1.items() if count >=
min_support_count}
frequent_itemsets = [L1]
k = 2

--- Step 3: Generate Frequent Itemsets ---
while True:
```

```

prev_Lk = list(frequent_itemsets[-1].keys())
candidates = []

for i in range(len(prev_Lk)):
 for j in range(i + 1, len(prev_Lk)):
 union = prev_Lk[i].union(prev_Lk[j])
 if len(union) == k and union not in candidates:
 subsets = list(combinations(union, k - 1))
 if all(frozenset(s) in prev_Lk for s in subsets):
 candidates.append(union)

Ck = {}
for candidate in candidates:
 count = count_support(candidate, transactions)
 if count >= min_support_count:
 Ck[frozenset(candidate)] = count

if not Ck:
 break

frequent_itemsets.append(Ck)
k += 1

--- Step 4: Show Final Frequent Itemsets ---
final_frequent = frequent_itemsets[-1]
print("\n Final Frequent Itemsets:")
for itemset in final_frequent:
 print(set(itemset))

--- Step 5: All Association Rules + Strong Rules ---
all_rules = []
strong_rules = []

print("\n All Association Rules with Confidence:")
for itemset, support_count in final_frequent.items():
 for i in range(1, len(itemset)):
 for lhs in combinations(itemset, i):
 lhs = frozenset(lhs)
 rhs = itemset - lhs
 if rhs:

```

```

 lhs_count = count_support(lhs, transactions)
 if lhs_count > 0:
 confidence = (support_count / lhs_count) * 100
 confidence = round(confidence, 2)
 print(f"{set(lhs)} => {set(rhs)} | confidence =
{confidence}%")

 all_rules.append((lhs, rhs, confidence))
 if confidence >= min_confidence_percent:
 strong_rules.append((lhs, rhs, confidence))

--- Step 6: Strong Association Rules ---
print("\n Strong Association Rules (confidence ≥ " +
str(min_confidence_percent) + "%):")
for lhs, rhs, confidence in strong_rules:
 print(f"{set(lhs)} => {set(rhs)} | confidence = {confidence}%")

```

### **g. FP Tree/Growth**

```
import pandas as pd
from itertools import combinations

--- Load Excel File ---
df = pd.read_excel(r'C:\Users\Cinepix\Downloads\Apriori.xlsx')

--- Preprocess Transactions ---
transactions = df['List of item Ids'].dropna().apply(lambda x:
set(str(x).strip().lower().split(' '))).tolist()
num_transactions = len(transactions)

--- User Input ---
min_support_percent = float(input("Enter minimum support %: "))
min_confidence_percent = float(input("Enter minimum confidence %: "))

--- Step 1: Minimum Support Count ---
min_support_count = round((min_support_percent * num_transactions) /
100)
print(f"\nMinimum Support Count: {min_support_count}")

--- Support Count Helper ---
def count_support(itemset, transactions):
 return sum(1 for t in transactions if itemset.issubset(t))

--- Step 2: Generate C1 and L1 ---
item_counts = {}
for transaction in transactions:
 for item in transaction:
 itemset = frozenset([item])
 item_counts[itemset] = item_counts.get(itemset, 0) + 1

C1 = item_counts
L1 = {itemset: count for itemset, count in C1.items() if count >=
min_support_count}
frequent_itemsets = [L1]
k = 2

--- Step 3: Generate Frequent Itemsets ---
while True:
```

```

prev_Lk = list(frequent_itemsets[-1].keys())
candidates = []

for i in range(len(prev_Lk)):
 for j in range(i + 1, len(prev_Lk)):
 union = prev_Lk[i].union(prev_Lk[j])
 if len(union) == k and union not in candidates:
 subsets = list(combinations(union, k - 1))
 if all(frozenset(s) in prev_Lk for s in subsets):
 candidates.append(union)

Ck = {}
for candidate in candidates:
 count = count_support(candidate, transactions)
 if count >= min_support_count:
 Ck[frozenset(candidate)] = count

if not Ck:
 break

frequent_itemsets.append(Ck)
k += 1

--- Step 4: Show Final Frequent Itemsets ---
final_frequent = frequent_itemsets[-1]
print("\n Final Frequent Itemsets:")
for itemset in final_frequent:
 print(set(itemset))

--- Step 5: All Association Rules + Strong Rules ---
all_rules = []
strong_rules = []

print("\n All Association Rules with Confidence:")
for itemset, support_count in final_frequent.items():
 for i in range(1, len(itemset)):
 for lhs in combinations(itemset, i):
 lhs = frozenset(lhs)
 rhs = itemset - lhs
 if rhs:

```

```

 lhs_count = count_support(lhs, transactions)
 if lhs_count > 0:
 confidence = (support_count / lhs_count) * 100
 confidence = round(confidence, 2)
 print(f"{set(lhs)} => {set(rhs)} | confidence =
{confidence}%")

 all_rules.append((lhs, rhs, confidence))
 if confidence >= min_confidence_percent:
 strong_rules.append((lhs, rhs, confidence))

--- Step 6: Strong Association Rules ---
print("\n Strong Association Rules (confidence ≥ " +
str(min_confidence_percent) + "%):")
for lhs, rhs, confidence in strong_rules:
 print(f"{set(lhs)} => {set(rhs)} | confidence = {confidence}%")

```



## **h. Linear Regression (Prediction Algorithm)**

```
import pandas as pd
import matplotlib.pyplot as plt

Step 1: Load Excel data
df =
pd.read_excel(r'C:\Users\Cinepix\Downloads\Linear_Regression.xlsx')

Step 2: Extract X and Y values (assuming 1st and 2nd columns)
X = df.iloc[:, 0].tolist()
Y = df.iloc[:, 1].tolist()

Step 3: Calculate Avg(X) and Avg(Y)
x_bar = sum(X) / len(X)
y_bar = sum(Y) / len(Y)

print(f"x_bar (Average of X) = {x_bar:.2f}")
print(f"y_bar (Average of Y) = {y_bar:.2f}")

Step 4: Calculate β (slope)
numerator = sum((x - x_bar) * (y - y_bar) for x, y in zip(X, Y))
denominator = sum((x - x_bar) ** 2 for x in X)
beta = numerator / denominator

Step 5: Calculate α (intercept)
alpha = y_bar - beta * x_bar

Step 6: Display equation
print(f"\n β (Slope) = {beta:.2f}")
print(f" α (Intercept) = {alpha:.2f}")
print(f"Linear Regression Equation: y = {alpha:.2f} + {beta:.2f}x")

Step 7: Plot the data and regression line
plt.figure(figsize=(8, 5))
plt.scatter(X, Y, color='blue', label='Data Points')

Step 8: Predict Y for a given X
input_x = float(input("Enter an X value to predict Y: "))
predicted_y = alpha + beta * input_x
```

```
print(f"Predicted Y for X = {input_x:.2f} is: {predicted_y:.2f}")

Regression line points
x_line = list(range(min(X), max(X) + 1))
y_line = [alpha + beta * x for x in x_line]

plt.plot(x_line, y_line, color='red', label='Regression Line')

Styling the plot
plt.title("Linear Regression")
plt.xlabel("X")
plt.ylabel("Y")
plt.legend()
plt.grid(True)
plt.show()
```

## i. Data Discretization & Visualization

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

Load data from Excel
df =
pd.read_excel(r'C:\Users\Cinepax\Downloads\DataDiscretization_Visualiz
ation.xlsx')
data = df.iloc[:, 0].values
data = sorted(data) # Returns a new sorted list

User input for number of bins
num_bins = int(input("Enter the number of bins: "))
bin_size = len(data) // num_bins

Create equi-depth bins
bins = [data[i * bin_size : (i + 1) * bin_size] for i in
range(num_bins - 1)]
bins.append(data[(num_bins - 1) * bin_size:])

Bin transformation - means
mean_bins = [np.full(len(b), np.mean(b)) for b in bins]

Bin transformation - boundaries
boundary_bins = [
 [min(b) if val < (min(b) + max(b)) / 2 else max(b) for val in b]
 for b in bins
]

== Cleanly Print the bin details ==
print("\nOriginal Bins (Equi-depth):")
for i, b in enumerate(bins, start=1):
 print(f"Bin {i}: {[int(val) for val in b]}")

print("\nBin Means:")
for i, b in enumerate(mean_bins, start=1):
 print(f"Bin {i}: {[round(float(val), 1) for val in b]}")
```

```
print("\nBin Boundaries:")
for i, b in enumerate(boundary_bins, start=1):
 print(f"Bin {i}: {[int(val) for val in b]}")

Flatten all bins for plotting
original = data
means = np.concatenate(mean_bins)
boundaries = np.concatenate(boundary_bins)

Plot histograms
titles = ["Original Data", "Bin Means", "Bin Boundaries"]
datasets = [original, means, boundaries]
colors = ["skyblue", "orange", "green"]

plt.figure(figsize=(10, 12))
for i in range(3):
 plt.subplot(3, 1, i+1)
 plt.hist(datasets[i], bins=num_bins, color=colors[i],
edgecolor='black')
 plt.title(titles[i])
 if i == 2: plt.xlabel("Value")
 plt.ylabel("Frequency")
 plt.grid(True)

plt.tight_layout()
plt.show()
```